Universidade do Minho

Ano Letivo: 2025/26

Bases de Dados

Modelação Lógica (continuação)
Normalização



Sumário

1 Normalização

2 Modelação Lógica - SQLDeveloper

Bibliografia:

- Connolly, T., Begg, C., Database Systems, A Practical Approach to Design, Implementation, and Management , Addison-Wesley, 4a Edição, 2004. **(Chapters 14 and 17)**
- Belo, O., "Bases de Dados Relacionais: Implementação com MySQL", FCA – Editora de Informática, 376p, Set 2021. ISBN: 978-972-722-921-5.

Teoria da Normalização

Normalização de Dados

A normalização de dados é um método usado para organizar a informação numa base de dados relacional. Ao aplicá-la, os atributos de um modelo de dados são estruturados de forma a manter coerência entre as entidades, reduzindo ou eliminando a repetição de dados. Este processo contribui para melhorar a eficiência no armazenamento, bem como a integridade e a capacidade de crescimento da base de dados. Nesta unidade de sistemas de bases de dados serão abordados os principais conceitos relacionados com a normalização, dando especial atenção ao processo de normalização, às diferentes formas normais, às dependências funcionais e, por fim, à possível "desnormalização" de uma base de dados.

> *"There is, in fact, a very simple elimination procedure which we shall call normalization. Through decomposition nonsimple domains are replaced by 'domains whose elements are atomic (nondecomposable) values.'"*
> (Codd, 1970)

Teoria da Normalização

Normalização de Dados

A normalização de dados baseia-se na análise das **chaves primárias** e das **dependências funcionais** de todos os seus atributos.

É um processo **progressivo**, que assenta na execução de uma série de etapas, cada uma delas correspondendo a uma **forma normal** específica com critérios de validação cada vez mais fortes.

Através da sua aplicação, os atributos de um dado modelo de dados são organizados para assegurar a **coesão** dos tipos das entidades envolvidas, minimizando ou mesmo **eliminando duplicação de dados**, **melhorando a eficiência de armazenamento**, a **integridade** e a **escalabilidade** dos dados.

Teoria da Normalização

Normalização de Dados

Num processo de normalização de dados, é importante ter em conta alguns princípios fundamentais:

- Cada registo de uma tabela deve representar uma **entidade** ou uma **instância de um relacionamento**.
- A ligação ou referência a outras entidades deve ser feita **apenas através de chaves estrangeiras**.
- Sempre que possível, os **atributos das entidades e os dos relacionamentos devem ser separados**.
- A estrutura do esquema da base de dados deve ser organizada de forma que **possa ser compreendida e explicada tabela a tabela**.
- A semântica dos atributos deve ser clara e fácil de compreender.
- Misturar atributos de diferentes entidades pode causar vários problemas, como aumento do espaço de armazenamento devido a informação redundante ou anomalias na atualização dos dados.

Teoria da Normalização

Normalização de Dados

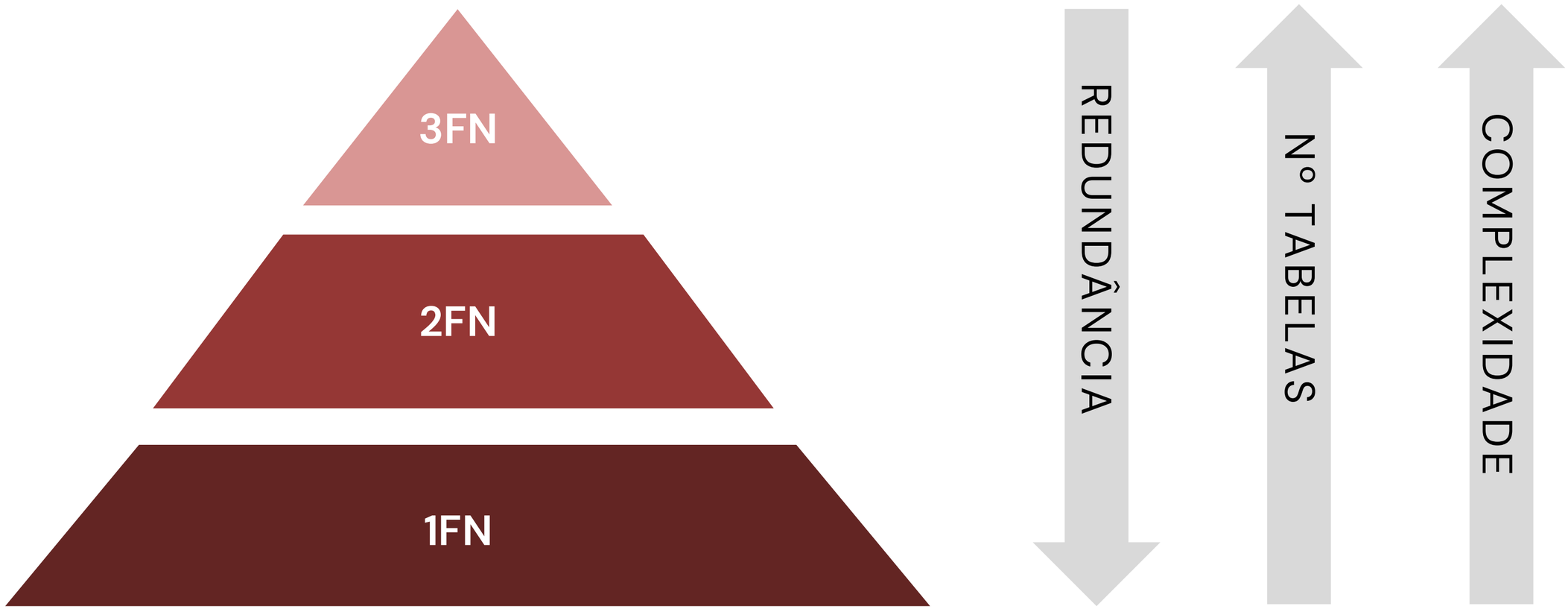
No **modelo relacional**, existem técnicas próprias para avaliar o nível de eficiência e organização de uma base de dados. Essas técnicas são conhecidas como **Formas Normais (FN)**. Para alcançar essas formas, existem algoritmos específicos que permitem transformar ou reorganizar uma base de dados, garantindo que ela cumpra os requisitos de cada forma normal.

- **1FN – Primeira Forma Normal.**
- **2FN – Segunda Forma Normal.**
- **3FN –Terceira Forma Normal.**
- FNBC – Forma Normal de Boyce-Codd.
- 4FN – Quarta Forma Normal.
- 5FN – Quinta Forma Normal.
- FNDC – Forma Normal Domínio-chave.
- 6FN – Sexta Forma Normal.

Teoria da Normalização

Formas Normais

O processo de normalização é **progressivo**, ou seja, cada um dos níveis superiores de normalização é um subconjunto do respetivo nível inferior.



Teoria da Normalização

➡ Primeira Forma Normal – 1FN

Diz-se que uma relação está na 1FN se:

1. Possuir uma chave primária.

2. Todos os seus atributos forem atómicos. Não são permitidos atributos que implicitamente codificam subatributos (atributos compostos) ou atributos multivalor.

3. Não possuir grupos de dados repetitivos.

Na prática, podemos dizer que uma relação está na 1FN se as interseções entre colunas (atributos) e linhas (registos) possuírem um único valor – um valor atómico.

Teoria da Normalização

➡ Primeira Forma Normal – 1FN

Aplicação da 1FN:

Passo 1: Uma das chaves candidatas é escolhida para chave primária.

Passo 2: Atributos multivalor são convertidos em novas relações com chave externa referindo a chave primária da tabela original.

Passo 3: Cada atributo composto é mapeado em vários sub-atributos atómicos.

Teoria da Normalização

➡ Segunda Forma Normal – 2FN

Diz-se que uma relação está na segunda forma normal (2FN) se:

1. A relação estiver também na 1FN.
2. Todos os seus atributos não-primos forem **totalmente dependentes** da sua chave primária. Isto é, não podem existir **dependências parciais**. Diz-se que um atributo é não-primo quando este não faz parte de uma chave primária.

Teoria da Normalização

➡ Segunda Forma Normal – 2FN

O que é uma dependência funcional?

As dependências funcionais determinam a forma como se pode interpretar e relacionar os dados e permitem especificar medidas formais sobre a correção dos esquemas relacionais.

Na prática, a dependência funcional **A1** → **A2** entre dois conjuntos de atributos de uma relação significa que:

- para cada valor de A1, existe apenas um valor possível para A2, por isso diz-se que A2 é funcionalmente dependente de A1 ou que A1 determina funcionalmente A2;

- valores iguais para A1, determinam valores iguais para A2.

Teoria da Normalização

Segunda Forma Normal – 2FN

Aplicando a análise de dependências funcionais:

Alunos(id_aluno, nome_aluno, cod_curso, nome_curso)

Notas(id_aluno, cod_dis, nome_dis, nota, cod_prof, nome_prof, dep_prof)

Diagrama de Dependências

id_aluno → nome_aluno, cod_curso

cod_curso → nome_curso

{id_aluno, cod_dis} → nota

cod_disc → nome_dis, cod_prof

cod_prof → nome_prof, dep_prof

A tabela Alunos está na 2FN, mas a Notas não!

Teoria da Normalização

Segunda Forma Normal – 2FN

Diagrama de Dependências

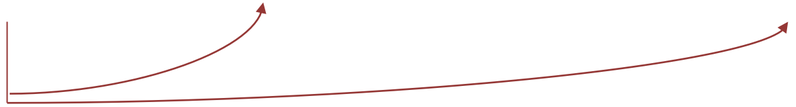
{id_aluno, cod_dis} → nota

cod_disc → nome_dis, cod_prof

cod_prof → nome_prof, dep_prof

Notas(<u>id_aluno, cod_dis</u>, nome_dis, nota, cod_prof, nome_prof, dep_prof)

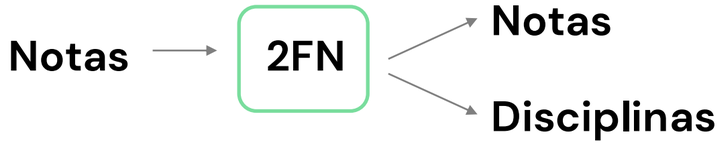
<u>id_aluno</u>	**<u>cod_dis</u>**	**nome_dis**	**nota**	**cod_prof**	**nome_prof**	**dep_prof**
001	D01	Bases de Dados	16	P01	Maria do Carmo	Dep. Informática
001	D02	Criptografia	12	P02	Paulo Gomes	Dep. Informática
002	D01	Bases de Dados	17	P01	Maria do Carmo	Dep. Informática
002	D03	Lógica Computacional	14	P03	Tiago Pinho	Dep. Sistemas
...	...	...	...	...	...	...



Dependência Parcial

Teoria da Normalização

➡ Segunda Forma Normal – 2FN



Disciplinas(<u>cod_dis</u>, nome_dis, cod_prof, nome_prof, dep_prof)

Notas
<u>id_aluno</u>
<u>cod_dis</u>
nota

(001, ‘D01’, 16)
(001,’D02’, 12)
(002,’D01’, 17)
(002,’D03’, 14)
…

Disciplinas
<u>cod_dis</u>
nome_dis
cod_prof
nome_prof
dep_prof

Teoria da Normalização

Terceira Forma Normal – 3FN

Diz-se que uma relação está na 3FN se:

1. A relação estiver também na 1FN e na 2FN.
2. Todos os seus atributos que não sejam chaves primárias sejam mutuamente independentes, não havendo assim **dependências funcionais transitivas**. Por outras palavras, numa relação na 3FN, todos os atributos dependem única e exclusivamente da chave primária.

Na prática, isto significa que os atributos que não dependam da chave primária devem ser "eliminados" da relação, ou seja, devem ser transferidos para outra tabela.

Teoria da Normalização

Terceira Forma Normal – 3FN

Na prática, a dependência funcional **A1** → **A2, A3** e **A3** → **A4** entre os atributos de uma relação significa que:

- Existe uma dependência funcional transitiva entre A1 e A4. Ou seja os atributos que não são chave primária, não são mutuamente independentes entre si.

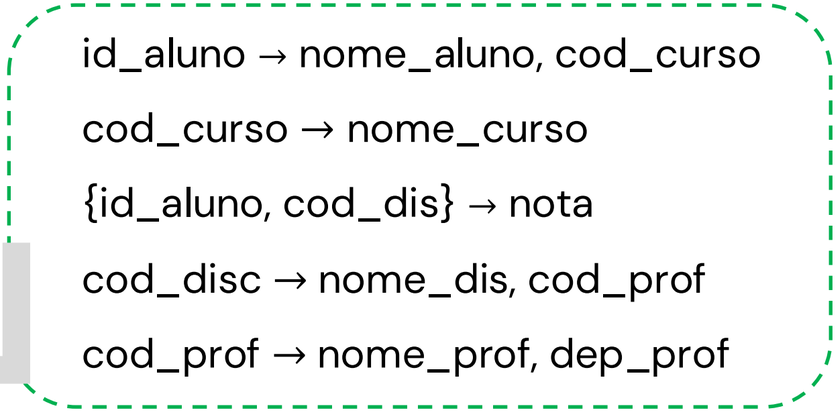
Aplicando a análise de dependências funcionais ao caso de estudo anterior:

Alunos(id_aluno, nome_aluno, cod_curso, nome_curso)

Notas(id_aluno, cod_dis, nota)

Disciplinas(cod_dis, nome_dis, cod_prof, nome_prof, dep_prof)

Diagrama de Dependências



A tabela Alunos e Disciplinas não estão na 3FN!

Teoria da Normalização

Terceira Forma Normal – 3FN

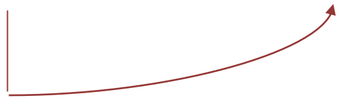
Diagrama de Dependências

id_aluno → nome_aluno, cod_curso

cod_curso → nome_curso

Alunos(id_aluno, nome_aluno, cod_curso, nome_curso)

id_aluno	nome_aluno	cod_curso	nome_curso
001	João Ferreira	C01	MIEI
001	João Ferreira	C01	MIEI
002	Rita Abreu	C01	MIEI
002	Rita Abreu	C01	MIEI
...	...	...	...



Dependência Transitiva

Teoria da Normalização

Terceira Forma Normal – 3FN

Diagrama de Dependências

cod_disc → nome_dis, cod_prof

cod_prof → nome_prof, dep_prof

Disciplinas(<u>cod_dis</u>, nome_dis, cod_prof, nome_prof, dep_prof)

<u>cod_dis</u>	nome_dis	cod_prof	nome_prof	dep_prof
DO1	Bases de Dados	PO1	Maria do Carmo	Dep. Informática
DO2	Criptografia	PO2	Paulo Gomes	Dep. Informática
DO1	Bases de Dados	PO1	Maria do Carmo	Dep. Informática
DO3	Lógica Computacional	PO3	Tiago Pinho	Dep. Sistemas
...	...	...	...	...



Dependência Transitiva

Teoria da Normalização

Terceira Forma Normal – 3FN





Alunos(<u>id_aluno</u>, nome_aluno, cod_curso)

Alunos
<u>id_aluno</u>
nome_aluno
cod_curso

Disciplinas(<u>cod_dis</u>, nome_dis, cod_prof)

Disciplinas
<u>cod_dis</u>
nome_dis
cod_prof

Cursos(cod_curso, nome_curso)

Cursos
<u>cod_curso</u>
nome_curso

Professores(<u>cod_prof</u>, nome_prof, dep_prof)

Professores
<u>cod_prof</u>
nome_prof
dep_prof

Teoria da Normalização

➡ Terceira Forma Normal – 3FN

Alunos(<u>id_aluno</u>, nome_aluno, cod_curso)

Alunos
<u>id_aluno</u>
nome_aluno
cod_curso

Disciplinas
<u>cod_dis</u>
nome_dis
cod_prof

Cursos(cod_curso, nome_curso)

Cursos
<u>cod_curso</u>
nome_curso

Diagrama de Dependências

id_aluno → nome_aluno, cod_curso

cod_curso → nome_curso

{id_aluno, cod_dis} → nota

cod_disc → nome_dis, cod_prof

cod_prof → nome_prof, dep_prof

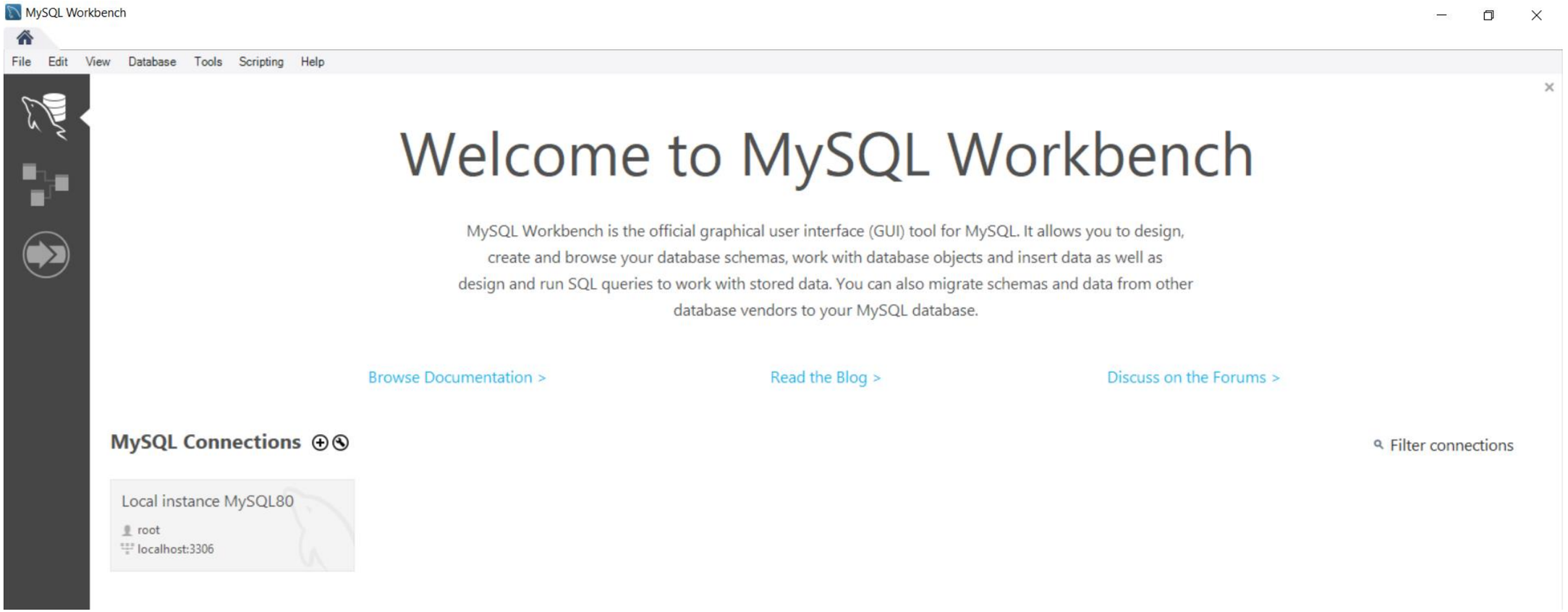
Disciplinas
<u>cod_prof</u>
nome_prof
dep_prof

As relações encontram-se na 3FN!

FASE 4: Modelação Lógica

MySQL Workbench

1) Após a instalação, o GUI vai abrir com a configuração ao MySQL server já efetuada (assinalado na figura)

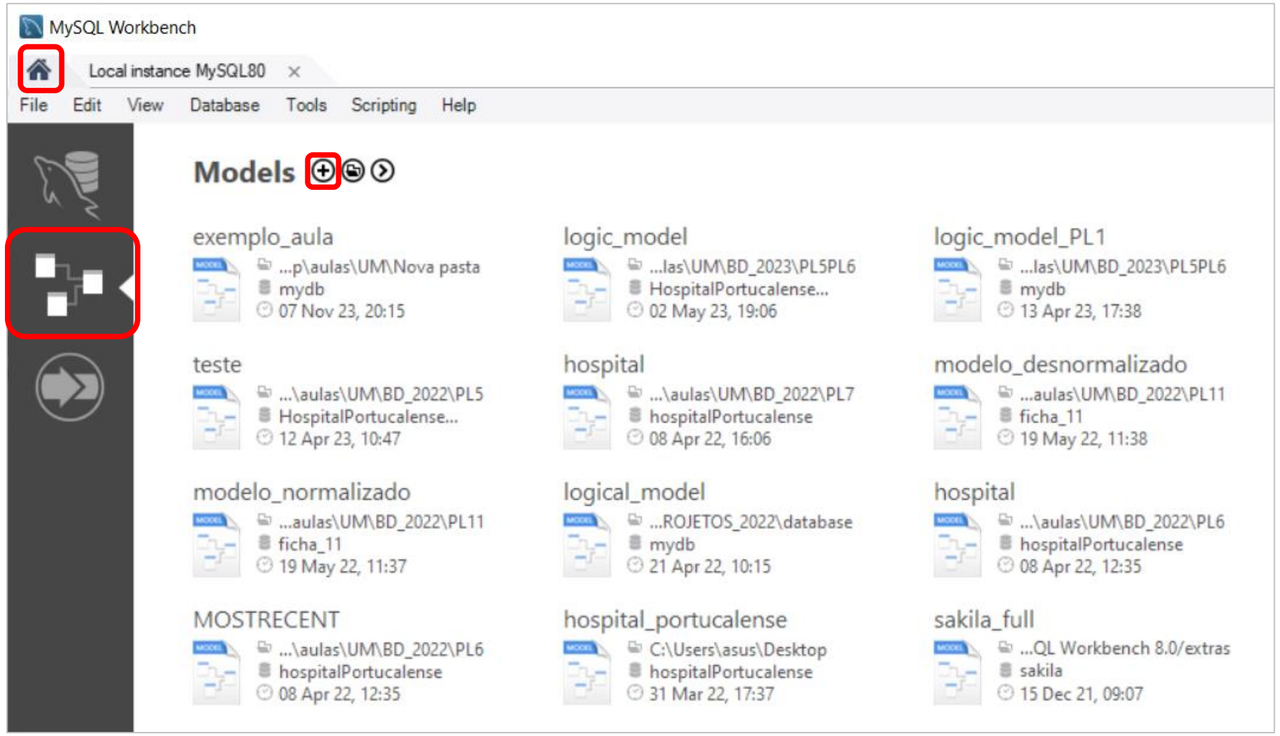


NOTA: se a conexão não aparecer, é provável que falte ou tenha falhado alguma etapa do guia de instalação

FASE 4: Modelação Lógica

➡ MySQL Workbench

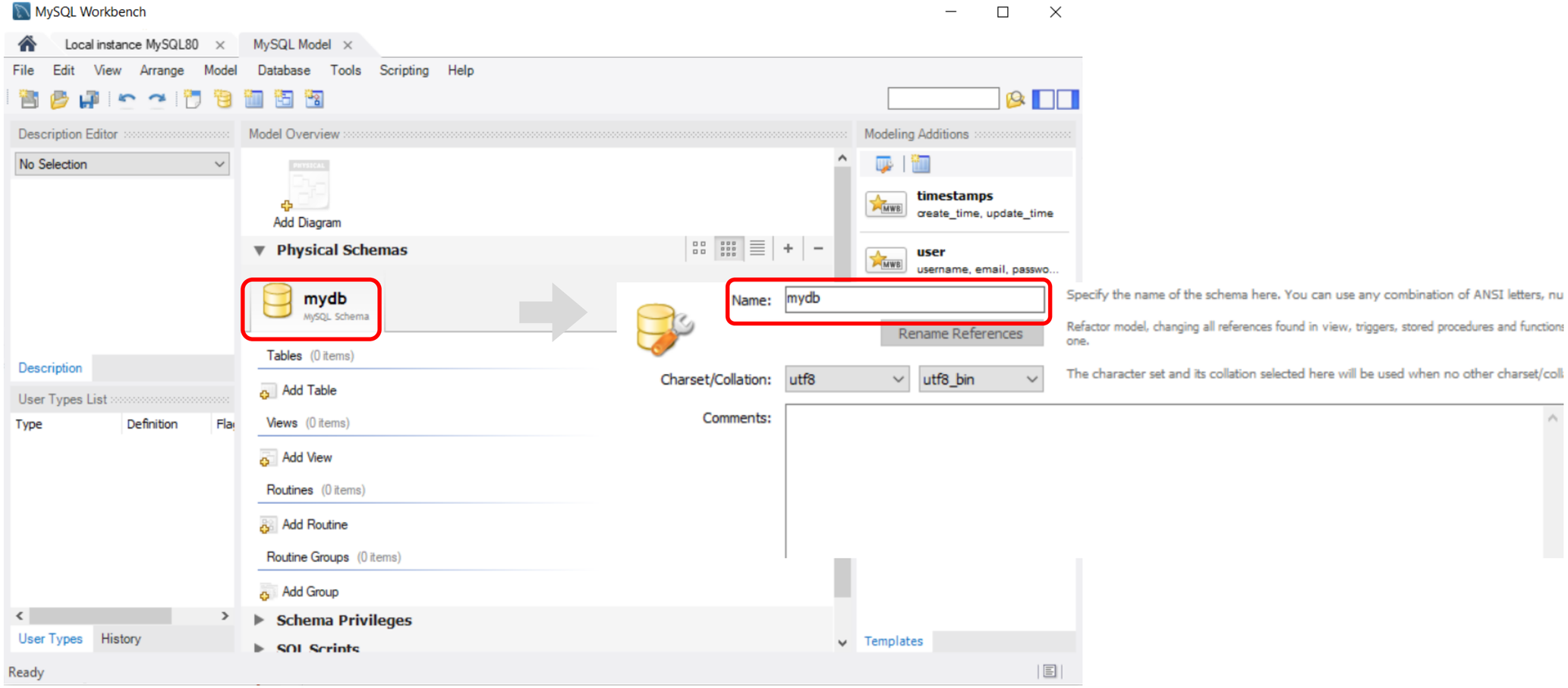
Ir ao menu inicial e clicar no separador “Models” para criar um novo esquema.



FASE 4: Modelação Lógica

MySQL Workbench

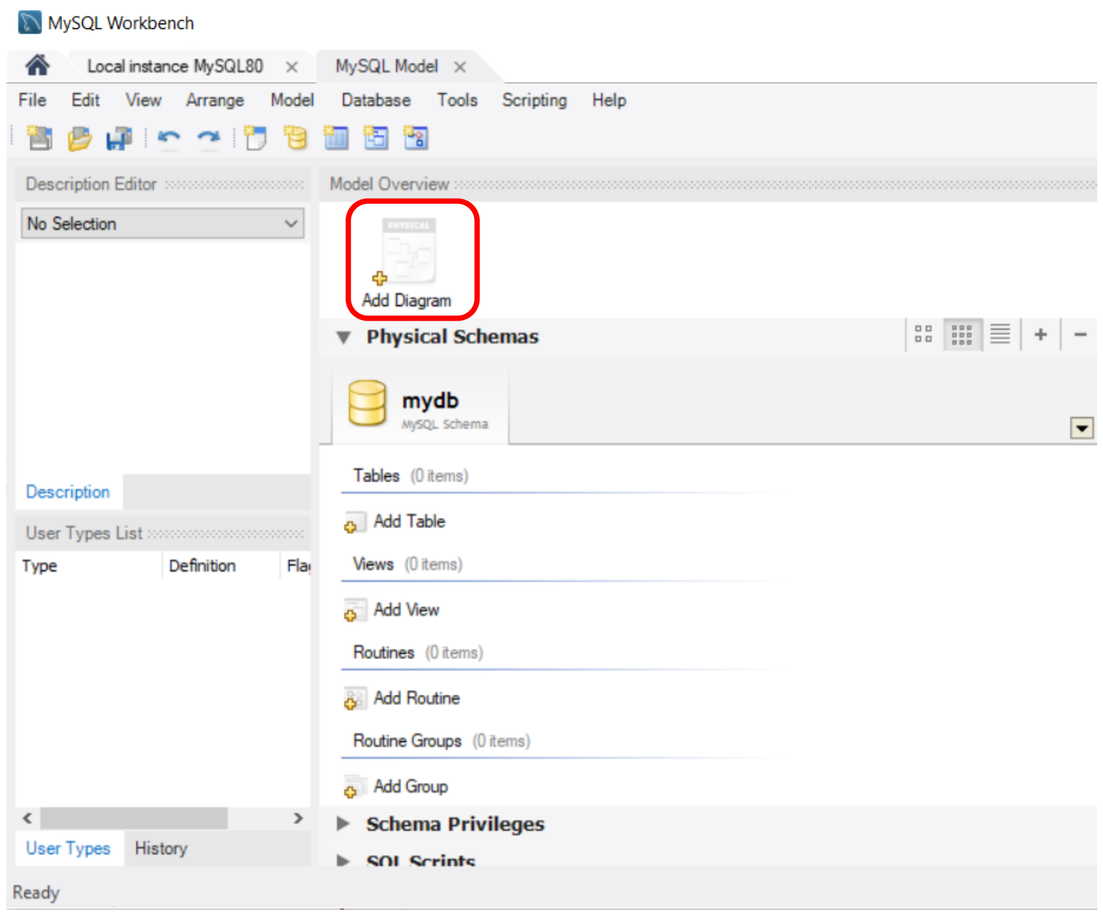
O Workbench cria então um novo esquema com o nome ‘mydb’. Para alterar o nome do esquema, basta clicar duas vezes em cima de ‘mydb’

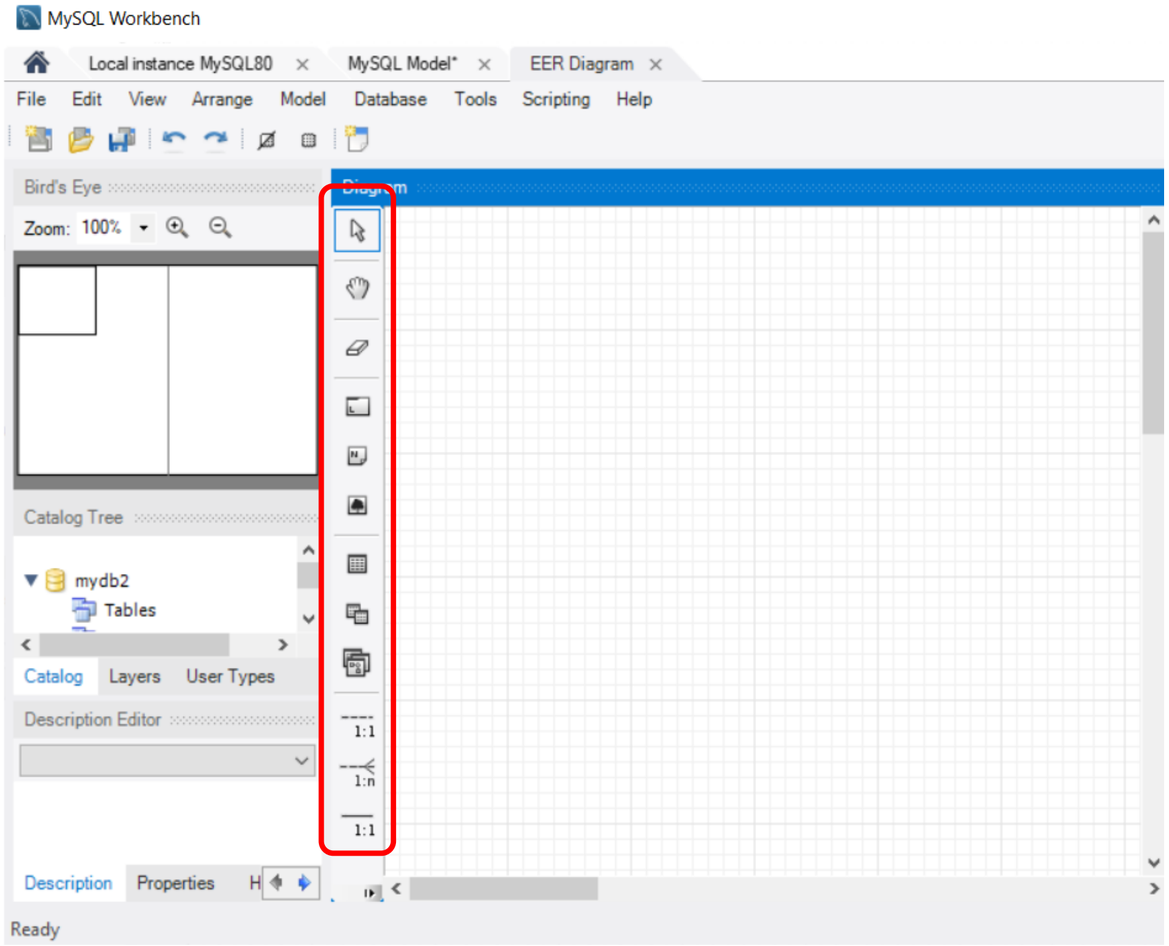


FASE 4: Modelação Lógica

➡ MySQL Workbench

Depois de configurar o nome, clicar no botão 'Add Diagram'. Uma nova janela é criada chamada 'EER Diagram'.





Modelação Lógica - MySQL

Quando estamos a construir o modelo lógico de dados no MySQL, é importante ter em consideração os seguintes aspetos:

- Tipo de relacionamento:



Relacionamentos identificadores (linha cheia) Quando a chave primária da entidade pai é incluída na chave primária da entidade filho.

- Chave estrangeira e chave primária.



Relacionamentos não identificadores (linha tracejada)
Quando a chave primária da entidade pai é incluída na entidade filho, mas não como parte da sua chave primária.

- Chave estrangeira NOT NULL – participação obrigatória no modelo conceptual
- Chave estrangeira – participação opcional no modelo conceptual

- Direcção do relacionamento:
  Os relacionamentos devem começar na relação/tabela que deve alocar a chave estrangeira.

Modelação Lógica - MySQL

- Valores padrão/por defeito: Devem ser usados caso se queira considerar um valor por *default*.



- PK (Primary Key), NN (Not Null), UQ (Unique Index), B (Binary), UN (Unsigned), ZF (Zero Fill), AI (auto increment), G (generated)

  - PK – deve ser usado para atributos que são chave primária;
  - NN – deve ser usado em todos os atributos de chave primária e todos os atributos que não possam ser NULL;
  - UQ - deve ser aplicado sempre que há chaves candidatas, faz com que não hajam valores duplicados na tabela;
  - UN - define que não podem ser inseridos valores negativos nessa coluna.
  - ZF - preenche o valor definido para o campo com zeros até a largura de exibição especificada na definição da coluna.
  - AI – deve ser usado para gerar automaticamente quando um novo registo é inserido numa tabela.
  - G – deve ser usado para gerar atributos a partir de outros usando uma expressão.

Tipos de Dados no MySQL

Dados Alfanuméricos

https://dev.mysql.com/doc/refman/8.0/en/data-types.html

VARCHAR (strings de tamanho **variável**) **vs. CHAR** (strings de tamanho **fixo**)

Valor	CHAR(4)		VARCHAR(4)	
''	'____'	4 bytes	''	1 byte
'AB'	'AB__'	4 bytes	'AB'	3 bytes
'ABC'	'ABC_'	4 bytes	'ABC'	4 bytes
'ABCD'	'ABCD'	4 bytes	'ABCD'	5 bytes

O VARCHAR usa 1 ou 2 bytes de memória adicionais para tamanho ou para marcar o fim dos dados.

Para armazenar textos mais longos:
- TEXT
- TINYTEXT
- MEDIUMTEXT
- LONGTEXT

Tipos de Dados no MySQL

Dados Alfanuméricos

https://dev.mysql.com/doc/refman/8.0/en/data-types.html

- O tipo **ENUM** é um objeto de string cujo valor é seleccionado a partir de um conjunto de valores permitidos que são definidos explicitamente no momento de criação da coluna.

EXEMPLO:

prioridade ENUM('Não Urgente', 'Pouco Urgente', 'Urgente', 'Muito Urgente', 'Emergente') NOT NULL);

A coluna prioridade aceitará apenas a inserção de um dos cinco valores definidos. O MySQL mapeia cada membro de enumeração para um índice numérico. Neste caso, 'Não Urgente', 'Pouco Urgente', 'Urgente', 'Muito Urgente' e 'Emergente' são mapeados para 1, 2, 3, 4 e 5 respectivamente.

- O tipo **SET** é um objeto string que pode ter zero ou mais valores, cada um dos quais deve ser escolhido a partir de um conjunto de valores especificados quando a tabela é criada.

EXEMPLO:

tipo SET('A', 'B') NOT NULL);

A coluna tipo aceitará a inserção de '', 'A', 'B' ou 'A,B'. O MySQL armazena valores SET numericamente, com o bit de ordem inferior do valor armazenado correspondendo ao primeiro membro do conjunto.

Tipos de Dados no MySQL

Dados de Data/Hora

https://dev.mysql.com/doc/refman/8.0/en/data-types.html

Tipo de Dados	Notação
DATE	YYYY-MM-DD
TIME	hh:mm:ss
DATETIME*	YYYY-MM-DD hh:mm:ss
TIMESTAMP**	YYYY-MM-DD hh:mm:ss
YEAR	YYYY

* O intervalo suportado varia de '1000-01-01 00:00:00' a '9999-12-31 23:59:59'.
** O intervalo suportado varia de '1970-01-01 00:00:01' a '2038-01-19 03:14:07'.

Tipos de Dados no MySQL

Dados de Data/Hora

https://dev.mysql.com/doc/refman/8.0/en/data-types.html

- Fixed-Point Types (Exact Value) - DECIMAL

Os tipos DECIMAL armazenam valores de dados numéricos exatos. Este tipo de dados é usado quando é importante preservar a precisão exata, por exemplo, com dados monetários.

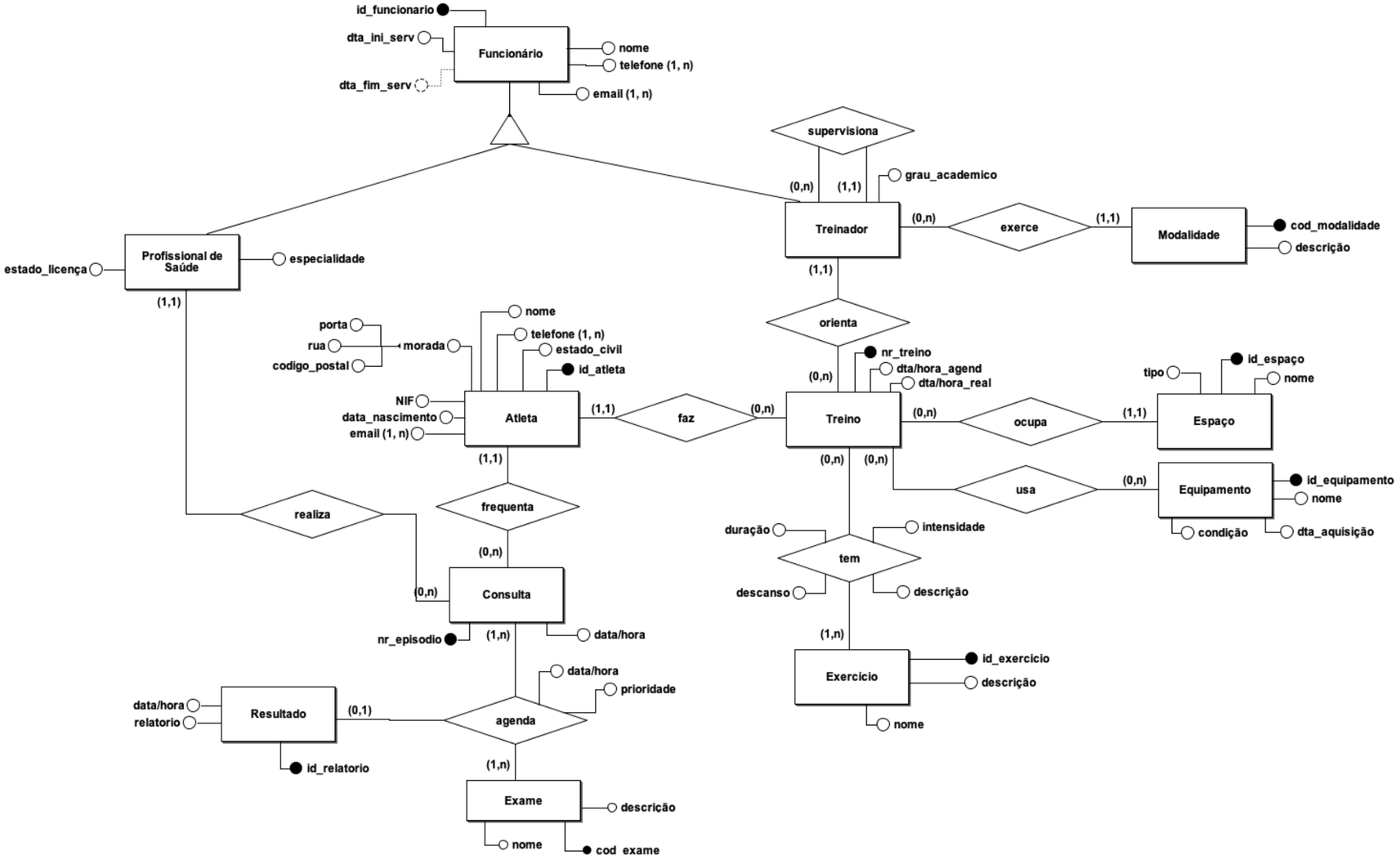
DECIMAL(n,m)

n – precisão - representa o número de dígitos significativos que são armazenados.
m – escala - representa o número de dígitos que podem ser armazenados após o ponto decimal.

Exemplo: 105,98€ -> DECIMAL (5,2)

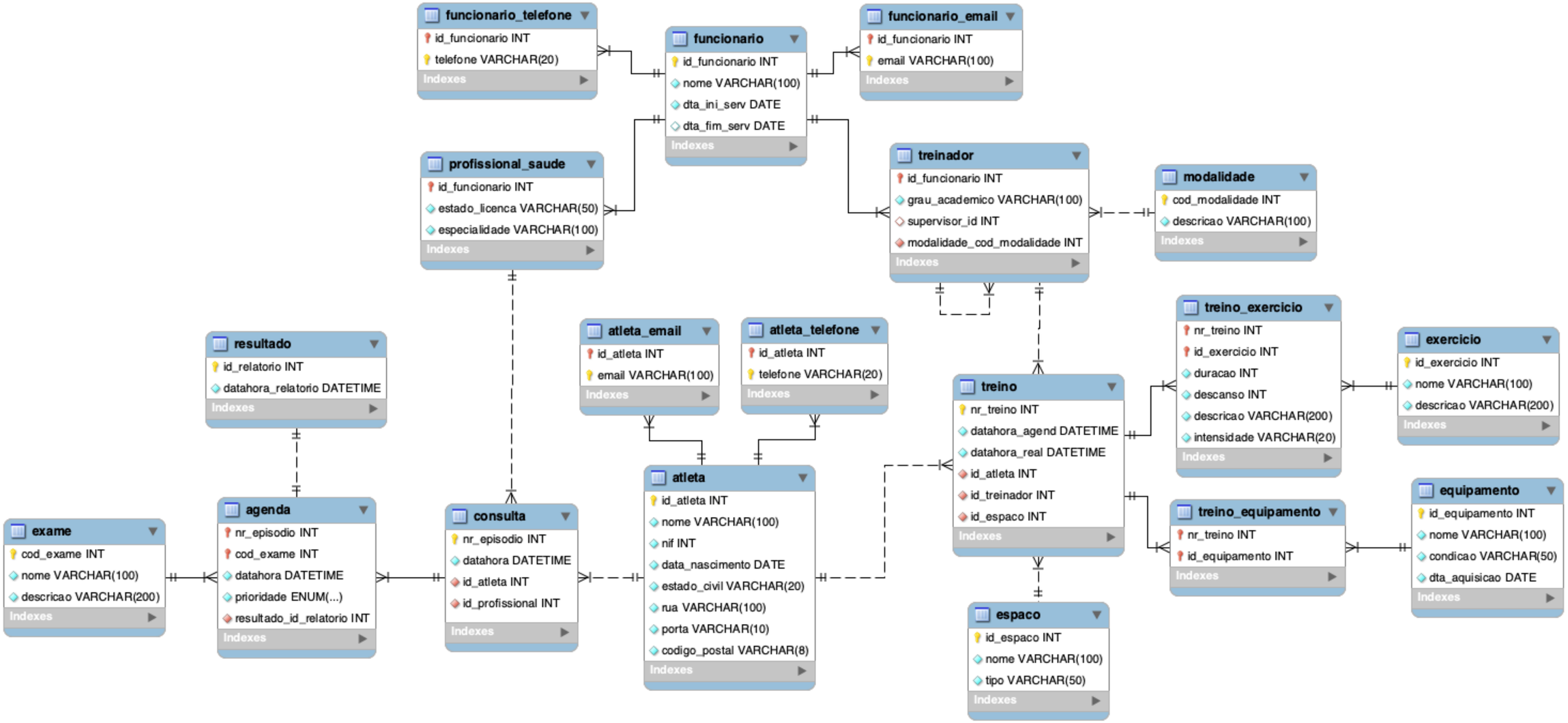
Modelação Lógica

Caso de Estudo



Modelação Lógica

➡ Caso de Estudo



Universidade do Minho

Ano Letivo: 2025/26

Bases de Dados

Modelação Lógica (continuação)
Normalização



António Abelha | abelha@di.uminho.pt